Day 103: Semantic Search with Embeddings — Real Vector-Similarity Search

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 15, Day 6 | Complete study guide

Watch: Lexical vs Semantic Search: BM25 vs Vectors (Kris Kassem) | Hybrid Search Finds What Vectors Miss (RAG) (Shane, LLM Implementation)

Script: `day103_semantic_search.py` · every number in this guide is copied by program from `day103_run_output.txt`, the real output of that script.

Four searches over all 2,067 SQuAD validation paragraphs, each asked to find the paragraph a question was written about. The result runs against the fashion: keyword search (BM25, from scratch) puts the right paragraph first for 0.758 of 1,000 questions, the pretrained sentence-embedding search for only 0.629. Dense search catches up only on questions that share few words with their paragraph, and combining the two (0.765) is barely better than BM25 alone. The reason is in how SQuAD was written: people read the paragraph, then wrote the question in its words.

0. Where today fits

DEFINITION — start here, no prior knowledge assumed

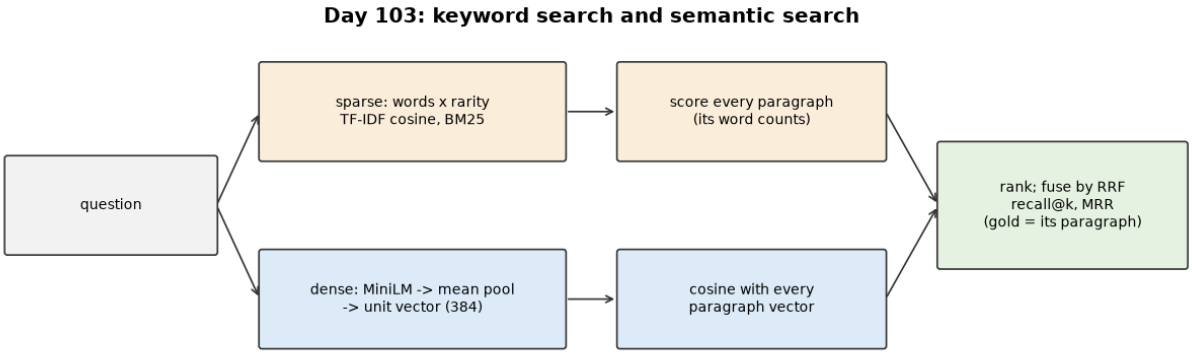
Keyword (sparse) search scores a document by the query words it contains, weighting rare words more: **TF-IDF** and **BM25**, which also stops rewarding repeats after a few and adjusts for document length.

Semantic (dense) search turns every text into a single vector with a model trained so that texts with similar meaning get vectors pointing the same way; the search returns the documents with the highest **cosine similarity** to the query's vector. It can match 'population' with 'residents'.

REAL-WORLD SCENARIO

A library catalogue that matches the words you type, against a librarian who understands what you mean even when you use different words.

SYSTEM DIAGRAM



Two routes from a question to a ranked list of paragraphs, and their fusion.

Today's part for Day 104

`day103_exercise.py` is the retriever part: `mean_pool`, `cosine_top_k` and `BM25`, each checked against a case with a known answer. Its real output:

ACTUAL OUTPUT

```
mean pooling with and without 3 padding tokens: largest difference 0.0e+00
top-5 for 20 queries over 500 vectors equals a full sort: True; scores descending: True
idf: banana (in 4 of 5 documents) 0.288, durian (in 2 of 5) 0.875
'apple' once vs three times: 0.924 vs 1.397 (three times is not three times the score)
'banana' in a 2-word vs a 6-word document: 0.353 vs 0.214 (the shorter one wins)
best document for 'durian banana': 'banana durian'
```

How to read each section

Step	What it gives you
DEFINITION	What the idea is, in plain words. Start here.
REAL-WORLD SCENARIO	An everyday situation where the same idea shows up.
MATHEMATICS	The formula, when the section introduces one.
IMPLEMENTATION	The real code from the script.
ACTUAL OUTPUT	What the script really printed, copied unchanged.
What it means	How to read the numbers, and what they do and do not show.
Why choose this? / Why NOT this?	When the idea is the right tool, and what it costs.

1. Embeddings: one vector per text

DEFINITION — start here, no prior knowledge assumed

A sentence-embedding model (all-MiniLM-L6-v2, 6 layers) gives a vector for every token. **Mean pooling** averages them, ignoring padding, into one 384-number vector per text; dividing by its length makes the dot product of two vectors equal their cosine similarity.

REAL-WORLD SCENARIO

Summarising a book by the average of all its pages' themes, then comparing books by how alike their averages are.

MATHEMATICS

$$v = \Sigma_t m_t h_t / \Sigma_t m_t \quad (m = \text{attention mask}), \quad \cos(q, d) = q \cdot d / (|q|\,|d|)$$

IMPLEMENTATION

```
def mean_pool(hidden, mask):
    m = mask.unsqueeze(-1).float()
    return (hidden * m).sum(1) / m.sum(1).clamp(min=1e-9)
def embed(tok, enc, texts, bs=32, max_len=256):
    out = []
    with torch.no_grad():
        for i in range(0, len(texts), bs):
            b = tok(texts[i:i + bs], padding=True, truncation=True, max_length=max_len,
                return_tensors="pt")
            v = mean_pool(enc(**b).last_hidden_state, b["attention_mask"])
            out.append(torch.nn.functional.normalize(v, dim=-1).numpy())
    return np.concatenate(out)
```

ACTUAL OUTPUT

```
sentence-transformers/all-MiniLM-L6-v2: 22,713,216 weights, 6 layers, vectors of 384
cosine similarities (rows and columns in this order):
  How many people live in Paris?                  1.000  0.730  0.831 -0.054
  What is the population of the French capital?   0.730  1.000  0.613 -0.020
  Paris is home to about two million residents.   0.831  0.613  1.000 -0.014
  The cat sat on the mat.                        -0.054 -0.020 -0.014  1.000
mean pooling by hand vs the [CLS] vector, cosine with each other: 0.534 (they are different summaries)
Actual output from this lesson's run.
```

What it means

The vectors capture meaning across different words: 'How many people live in Paris?' is 0.831 similar to 'Paris is home to about two million residents' and 0.730 to 'What is the population of the French capital?', which shares no content word with it, while 'The cat sat on the mat' is near 0 (-0.054). Mean pooling and the [CLS] vector are different summaries (cosine 0.534); this model was trained with mean pooling, so that is the one to use.

WHY CHOOSE THIS?

Matches paraphrases and synonyms that share no words.

WHY NOT THIS? (TRADEOFFS)

Every document must be encoded once by a neural network; long ones are cut.

2. Four searches over every paragraph

DEFINITION — start here, no prior knowledge assumed

TF-IDF cosine, BM25 (k_1 = 1.5, b = 0.75), dense MiniLM cosine, and a **hybrid**: reciprocal rank fusion (RRF), which adds 1 / (60 + rank) from each list, so a paragraph ranked high by either search rises.

REAL-WORLD SCENARIO

Two shop assistants each list their best guesses; you go first to the shelf both put near the top.

MATHEMATICS

$$BM25(q, d) = \Sigma_{w \text{ in } q}\ idf(w) \cdot f(w,d)(k_1+1) / (f(w,d) + k_1(1 - b + b|d|/avgdl))$$

$$idf(w) = log(1 + (N - n_w + 0.5)/(n_w + 0.5)),\quad RRF(d) = \Sigma_s\ 1/(60 + rank_s(d))$$

IMPLEMENTATION

```
class BM25:
    def __init__(self, docs, k1=1.5, b=0.75):
        self.tf = [Counter(words(d)) for d in docs]
        self.len = np.array([sum(t.values()) for t in self.tf])
        self.avg, self.k1, self.b, self.N = self.len.mean(), k1, b, len(docs)
        df = Counter(w for t in self.tf for w in t)
        self.idf = {w: np.log(1 + (self.N - n + 0.5) / (n + 0.5)) for w, n in df.items()}
    def scores(self, query):
        s = np.zeros(self.N)
        norm = self.k1 * (1 - self.b + self.b * self.len / self.avg)
        for w in set(words(query)):
            if w in self.idf:
                f = np.array([t.get(w, 0) for t in self.tf])
                s += self.idf[w] * f * (self.k1 + 1) / (f + norm)
        return s
```

ACTUAL OUTPUT

```
collection: 2,067 paragraphs from 48 articles, 123 words on average; queries: 1,000 questions; the right answer is the
    paragraph each was written about
embedding: paragraphs 75 s, questions 4.2 s (4 threads); paragraphs longer than the encoder's 256-token limit, so cut: 168
BM25 scoring of all questions: 3.2 s (pure Python loops)
Actual output from this lesson's run.
```

What it means

Encoding all paragraphs took 75 s on this CPU; 168 paragraphs were longer than the encoder's 256-token limit and were cut, so their ends are invisible to dense search (a handicap BM25 does not have). BM25 in pure Python loops still scored all 1,000 questions in 3.2 s.

WHY CHOOSE THIS?

Sparse search is cheap and needs no model; dense search needs one encoding pass, then fast matrix products.

WHY NOT THIS? (TRADEOFFS)

Both rank every paragraph per question here; large collections need indexes (inverted lists, approximate neighbours).

3. Recall@k and MRR

DEFINITION — start here, no prior knowledge assumed

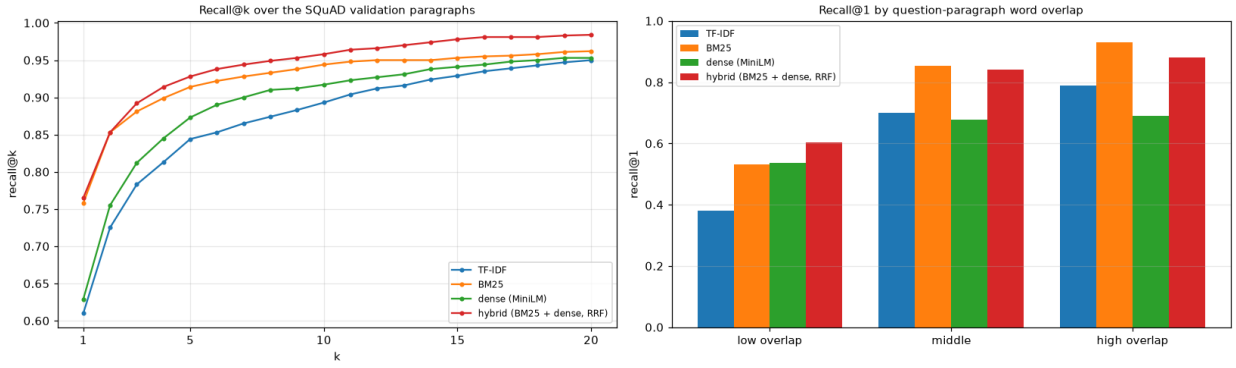
Recall@k: the share of questions whose paragraph is among the top k. **MRR**: the mean of 1/rank (1 for first place, 0.5 for second). Also counted: whether the first result at least comes from the right article.

REAL-WORLD SCENARIO

Did the right book make it onto the first shelf you checked, and how far down was it?

ACTUAL OUTPUT

```
system                          R@1     R@5    R@10     MRR     R@1 95% interval  same article @1
TF-IDF                        0.610   0.844   0.893   0.712      [0.579, 0.641]            0.883
BM25                          0.758   0.914   0.944   0.828      [0.732, 0.784]            0.946
dense (MiniLM)                0.629   0.873   0.917   0.735      [0.599, 0.658]            0.921
hybrid (BM25 + dense, RRF)    0.765   0.928   0.958   0.837      [0.738, 0.790]            0.961
paired differences in recall@1 (Day 99): mean, 95% interval, share of resamples ahead
  dense (MiniLM)              minus BM25             -0.129 [-0.161, -0.097] ahead in 0.000
  hybrid (BM25 + dense, RRF)  minus BM25             +0.007 [-0.018, +0.031] ahead in 0.701
  hybrid (BM25 + dense, RRF)  minus dense (MiniLM)   +0.136 [+0.111, +0.162] ahead in 1.000
  BM25                        minus TF-IDF           +0.148 [+0.123, +0.172] ahead in 1.000
Actual output from this lesson's run.
```



Left: recall@k for k = 1 to 20. Right: recall@1 by how many of the question's words appear in its paragraph.

What it means

BM25 beats dense search clearly at the top: 0.758 against 0.629 at rank 1, a difference of -0.129 with an interval far from 0. TF-IDF, the simpler keyword method, is the worst (0.610): BM25's saturation and length correction are worth +0.148. The hybrid is best in every column of this table (recall@10 0.958), but its lead over BM25 at rank 1 (+0.007) is within noise. When dense search misses, it usually still lands in the right article (0.921).

WHY CHOOSE THIS?

Recall@k says what a reader downstream will actually be given.

WHY NOT THIS? (TRADEOFFS)

One gold paragraph per question; another paragraph may also answer it.

4. When does dense search help?

DEFINITION — start here, no prior knowledge assumed

Questions split into thirds by how many of their (non-stop) words appear in their own paragraph.

REAL-WORLD SCENARIO

A librarian who understands you matters most when you cannot remember the book's exact words.

ACTUAL OUTPUT

```
share of a question's (non-stop) words that appear in its paragraph: mean 0.675; 0.146 of questions have every word there
overlap third              questions          TF-IDF            BM25  dense (MiniLM)  hybrid (BM25 +
low (0.00-0.60)                  373           0.381           0.531           0.536           0.603
middle (0.62-0.78)               301           0.701           0.854           0.678           0.841
high (0.80-1.00)                 326           0.788           0.929           0.690           0.880
(recall@1 in each third)
Actual output from this lesson's run.
```

What it means

On average 0.675 of a question's words are in its paragraph: SQuAD's writers read the paragraph first, and it shows. That favours keyword search: in the high-overlap third BM25 reaches 0.929, dense 0.690. In the low-overlap third the two are level (0.531 and 0.536) and the hybrid is best (0.603). With real users, who do not see the document first, overlap would likely be lower and dense search more useful; this dataset cannot show that.

WHY CHOOSE THIS?

Explains the headline result instead of just reporting it.

WHY NOT THIS? (TRADEOFFS)

Thirds of about 330 questions: differences of a few points are noise.

Interview preparation — technical questions

Q: What is mean pooling?

A: Averaging token vectors weighted by the attention mask, to get one vector per text.

Q: Why normalise embeddings?

A: Then a dot product equals cosine similarity, and top-k is one matrix product.

Q: What does BM25 add to TF-IDF?

A: Saturation of repeated words and a document-length correction: +0.148 recall@1 here.

Q: What is reciprocal rank fusion?

A: Sum of 1/(60 + rank) across systems: combines rankings without comparing raw scores.

Q: What are recall@k and MRR?

A: Share with the right answer in the top k; mean of 1/rank.

Interview preparation — theoretical questions

Q: Why did BM25 beat dense search here?

A: SQuAD questions reuse the paragraph's words (0.675 overlap on average).

Q: When is dense search better?

A: When queries paraphrase documents; here it tied BM25 on low-overlap questions.

Q: Why combine both?

A: They fail on different questions; the hybrid led on every overall measure, narrowly, though BM25 won the high-overlap questions.

Q: What happens to long documents in dense search?

A: They are cut at the encoder's limit (168 here) or must be chunked, as Day 104 does.

Key takeaway and what's next

> **KEY TAKEAWAY**
>
> Semantic search is a pretrained encoder, mean pooling and a matrix product. On SQuAD it lost to a from-scratch BM25 (0.629 against 0.758 at rank 1), most likely because the questions borrow the paragraphs' words; it caught up only on low-overlap questions, and fusing the two was the safest choice. Measure retrieval on your own queries before assuming embeddings win.

- Recall@1: TF-IDF 0.610, BM25 0.758, dense 0.629, hybrid 0.765.
- Recall@10: hybrid 0.958; dense's first result from the right article 0.921.
- Low-overlap third: BM25 0.531, dense 0.536, hybrid 0.603.
- 168 of 2,067 paragraphs cut at the encoder's 256 tokens.

Suggested watch order, if starting from zero

- 1. Lexical vs Semantic Search: BM25 vs Vectors (Kris Kassem) — keyword search against vector search.
- 2. Hybrid Search Finds What Vectors Miss (RAG) (Shane, LLM Implementation) — why hybrid search, and reciprocal rank fusion.

Day 104 preview

- **Week 15's build: a small RAG system.** Today's retrievers over passages cut from the same paragraphs, Day 102's reader and a pretrained generator, assembled from the week's exercise parts and measured end to end.